AI-Based Plant Disease Detection Using Deep Learning and Transfer Learning

Srisanth S
UG Student
Register No.: 24619151
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Vasantharaman S
UG Student
Register No.: 24619134
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Venkataramanan R
UG Student
Register No.: 24619156
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Berlin B
UG Student
Register No.: 24619139
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Dr. G. Sangeetha
Assistant Professor
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Abstract—Plant diseases affect how well crops grow and how good the harvest is. Finding them early and correctly lets farmers act before a disease spreads, but the usual method, visual inspection by farmers or plant experts, is slow and needs specialised knowledge. This paper presents an AI-based plant disease detection system that classifies plant leaf images using deep learning and transfer learning. The system uses the PlantVillage dataset with 38 classes and a MobileNetV2 network whose ImageNet pretrained weights are reused as a frozen feature extractor, followed by Global Average Pooling, Dropout and a softmax classification layer. Images are resized to 224 × 224 pixels and data augmentation is applied during training. The trained model is integrated into a web application built with Python, Flask, HTML, CSS and JavaScript, where a user uploads a leaf image and receives the predicted class with a confidence score. After 10 epochs of training, the model reached approximately 92.92% training accuracy and 94.11% validation accuracy. These are training and validation figures and are not independent test accuracy; a separate test-set evaluation is still required. The work shows how a deep learning model can be combined with a web application to build a plant disease detection tool.

Index Terms—Artificial Intelligence, Plant Disease Detection, Deep Learning, Transfer Learning, MobileNetV2, Convolutional Neural Network, PlantVillage, TensorFlow, Flask

I. Introduction

Agriculture plays a role in growing food and supporting national economies. Plant diseases can hurt how well crops grow, how much they produce and how good they look. These diseases often show up as spots, colour changes, sores, wilting or other clear signs on plant leaves, and catching them early helps avoid losing crops.

Traditionally, plant diseases are identified by looking at the plants. Farmers or experts check the leaves, compare the signs to known diseases and make a judgement. This works well when done by someone with a lot of experience, but it takes a long time and is hard to apply when many plants must be checked at once. Digital cameras and smartphones now make it possible to use computer vision and artificial intelligence to help automate this task.

Deep learning is one way to classify images. Convolutional Neural Networks (CNNs) can learn which features to look for in images and then group the images into categories. Training a deep CNN from the beginning, however, needs a very large dataset and powerful computers. Transfer learning offers a way around this by using a model that has already learned features from a large set of images. In this work MobileNetV2 is chosen as the base model and is adapted to recognise plant leaf images and sort them into the disease categories found in the dataset.

The trained model is combined with a Flask web application. A user uploads a photo of a plant leaf through the web interface, the image is sent to the model, and the predicted disease category and confidence score are shown to the user.

The main objectives of this work are:

1) To develop an AI-based plant disease classification system.
2) To use transfer learning for plant leaf image classification.
3) To classify images into plant disease categories.
4) To develop a web interface for image-based prediction.
5) To integrate the trained deep learning model with a Flask backend.
6) To provide prediction results with a confidence value.

II. Related Work

Computer vision and deep learning are being used more and more in farming to help farmers find problems in their crops. Earlier, detecting plant disease needed people to design image processing techniques that looked at colour, texture, shape and the appearance of a leaf. That work was difficult because

the techniques had to be hand-designed and the images had to be prepared carefully.

With the growth of CNN-based learning, it became possible to obtain features directly from plant images [1]. Instead of telling the model what to look for, the model learns the visual patterns from many training images, which makes the process easier and quicker.

Transfer learning makes deep learning more useful for image classification. Pretrained networks such as MobileNet [3], MobileNetV2 [2], ResNet and EfficientNet can be used for plant disease classification, and using them saves a lot of training time compared with creating a network from the beginning.

PlantVillage [4] is a dataset for plant disease classification. It contains images from many crops and many disease types. In this work the PlantVillage images are used to train the classification model and to check how well it performs.

III. Dataset and Preprocessing

A. Dataset

The system uses the PlantVillage dataset. It contains images belonging to a variety of plant species and disease categories. In the implemented system the dataset is arranged into folders that represent each class, and 38 classes are used. Because each class is stored in its own directory, TensorFlow's image dataset loader assigns the class labels automatically. The images are shuffled randomly and divided into three subsets, as shown in Table 1.

Table 1. Dataset Split

Subset	Percentage	Purpose
Training	70%	Learn model parameters
Validation	15%	Monitor performance during development
Testing	15%	Reserved for final evaluation

B. Image Preprocessing

The dataset is placed in training, validation and testing directories before training begins. All images are resized to 224 × 224 pixels, which is the input size required for MobileNetV2 in this work. RGB images are converted into arrays before being given to the neural network, and the MobileNetV2 preprocessing function from Keras is applied.

C. Data Augmentation

Data augmentation is applied during training to create varied versions of the training images. The transformations used are random flipping, rotation, random zooming and contrast adjustment. Augmentation helps the model learn features that do not depend too much on the exact appearance of each training image, and because it is applied during training it keeps changing the data the model sees.

IV. Proposed Methodology

A. System Overview

The proposed system is a web application that detects plant diseases from leaf images. It has two parts. The machine learning part loads the dataset, creates the training and testing data, processes the images, applies data augmentation, trains the MobileNetV2-based model, saves it and makes predictions about disease types. The web application part provides the user interface, receives plant leaf images, sends them to the Flask server, runs the trained model and shows the predicted result with its confidence level. The overall flow is shown in Fig. 1.

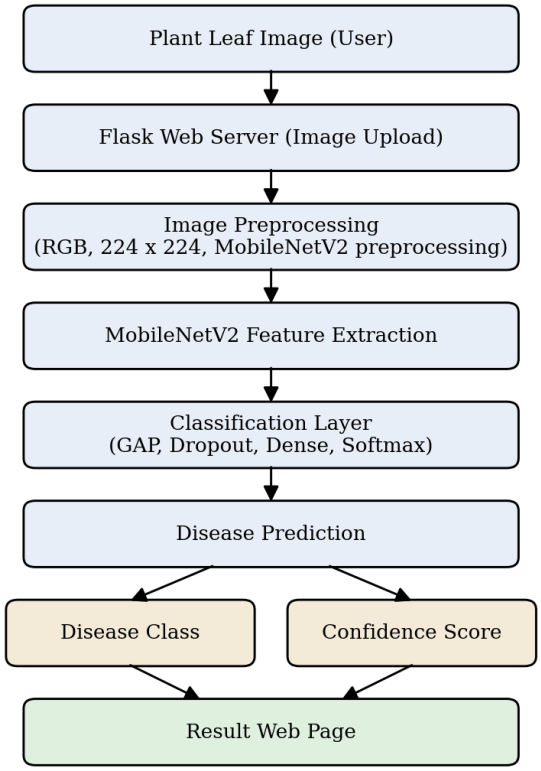


Figure 1. System architecture of the plant disease detection system.

B. MobileNetV2 Architecture

MobileNetV2 is a CNN architecture suited to image recognition [2]. Here it is loaded with ImageNet pretrained weights, and the base model is first frozen so that its pretrained feature representations are used without changing the weights during training. The model structure is: input image (224 × 224 × 3), data augmentation, MobileNetV2, Global Average Pooling, Dropout, a Dense classification layer and softmax over 38 classes.

- **Global Average Pooling** turns the feature maps produced by MobileNetV2 into a single feature set.
- **Dropout** is placed before the classification layer to prevent overfitting during training.
- **Dense layer** is the last layer and has one output for each plant disease class. Its softmax activation produces a probability distribution over the classes.

C. Transfer Learning

Transfer learning is a core part of the proposed system. MobileNetV2 is trained with pretrained weights from ImageNet: the ImageNet pretrained model acts as the MobileNetV2 feature extractor, the plant disease dataset is passed through it, and a new classification layer produces the plant disease prediction. This lets the system use features that were already learned from a large image dataset, and it reduces the resources needed compared with training a CNN from the beginning.

D. Model Prediction

After training, the saved model is loaded by the prediction module. When a user uploads an image, the following steps take place:

1) Flask receives the image file.
2) The file is checked to ensure it matches the allowed formats (JPG, JPEG and PNG).
3) The image is stored temporarily in the uploads folder.
4) The image is converted to RGB format.
5) The image is resized to 224 × 224 pixels.
6) MobileNetV2 preprocessing is applied.
7) The image is passed into the trained model, which returns probability scores for each class.
8) The class with the highest probability is selected, and the prediction and confidence value are sent back to the web application.

V. System Implementation

The web application is developed using Python, Flask, HTML, CSS, JavaScript and TensorFlow/Keras. Flask manages the backend and decides how the application responds to what users do, and each route is connected to a function that performs a task. The main routes are listed in Table 2.

Table 2. Main Flask Routes

Route	Function
/	Home page; welcomes users and describes the purpose of the application
/scan	Plant leaf scanning page where users upload a leaf image
/predict	Receives the image, sends it to the model and shows the prediction result

The project is organised into the following main files and folders. The file app.py runs the Flask application, handles web requests and connects the front end to the prediction system. The file train.py loads the training and validation datasets, builds the MobileNetV2-based model, trains it and saves it to disk. The file predict.py loads the trained model and the class names file, processes an image and returns the prediction. The file prepare_dataset.py prepares the dataset. The model folder stores plant_disease_model.keras and class_names.json, the data folder holds the train, validation and test subsets, the templates folder holds the home, upload and result pages (index.html, scan.html and result.html), and the static folder holds style.css and the uploads folder.

The web application offers the following path to the user. On the home page the user can reach the scanning feature. On the upload page the user uploads a leaf image in JPG, JPEG or PNG format. The image is sent to the Flask backend, which uses the deep learning model to examine it. The result page then shows the image, the predicted disease or class and the confidence percentage.

VI. Experimental Results

The model was trained on the PlantVillage dataset for 10 epochs with MobileNetV2 as the pretrained feature extractor. The initial training run achieved the results in Table 3.

Table 3. Training Results

Metric	Result
Training Accuracy	92.92%
Validation Accuracy	94.11%

These numbers are training and validation results and must not be reported as the real-world accuracy of the system. The 15% test subset has been reserved for evaluation, but a test-set evaluation has not yet been carried out. A complete evaluation should report accuracy, precision, recall, F1-score and a confusion matrix on the test set before performance claims are made.

VII. Discussion and Limitations

The results show that MobileNetV2-based transfer learning can learn features that help classify plant diseases. However, good validation accuracy does not guarantee that every picture taken in real life will be classified correctly. Images in the dataset differ from real-life pictures in several ways: lighting conditions, backgrounds, camera quality, leaf position, leaf size, image blur, multiple leaves in one picture, stages of disease and obstruction. Because of these differences, the model must be examined on the test dataset and also on real-life pictures, and tested on each group.

The current system has the following limitations:

1) **Dataset dependency.** Performance depends strongly on the quality and variety of the training dataset.
2) **Real-world image variation.** Images taken with phones may differ a lot from the controlled images in the dataset.
3) **Limited classes.** The current implementation contains 38 classes, and diseases outside these classes cannot be reliably classified.
4) **Confidence interpretation.** The softmax confidence value is the model's predicted probability distribution and should not automatically be taken as a guarantee that the prediction is correct.
5) **Image quality.** Blurred, poorly illuminated or heavily obstructed leaf images may reduce the reliability of the prediction.

VIII. Conclusion and Future Work

This paper presented an AI-based plant disease detection system that combines a MobileNetV2 transfer learning model with a Flask web application. The system automates plant disease classification, reduces the need for completely manual image inspection, supports multiple plant disease categories and provides a simple web-based interface. Because it uses the lightweight MobileNetV2 architecture, it can be extended with additional classes and could potentially be deployed as part of a larger agricultural decision-support application. The model reached approximately 92.92% training accuracy and 94.11% validation accuracy, and the next step is a proper test-set and real-image evaluation.

Several improvements are possible:

- **Fine-tuning.** The end layers of MobileNetV2 can be adjusted using a learning rate so that the pretrained features fit plant disease images better.
- **Better dataset.** Real pictures of plants can be collected to help the system work better in real use.
- **More plant types.** More kinds of plants and diseases can be added to the dataset.
- **Top three results.** Instead of one result, the application can display the top three possible classes.
- **Disease details.** The application can give a description of the illness, symptoms to watch for, and ways to stop it from spreading and to treat it.
- **Mobile application.** The model can be installed in an Android or other mobile application.
- **Online hosting.** The Flask application can be hosted on a server so that people can use it from anywhere.
- **Instant checking.** A later version might use a camera to check leaves in the field.

Acknowledgment

The authors thank their guide Dr. G. Sangeetha and the Department of Artificial Intelligence and Machine Learning, VISTAS, Chennai, for guidance and support throughout this project.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Front. Plant Sci.*, vol. 7, 2016.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.

[3] A. G. Howard *et al.*, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.

[4] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, 2015.